Tunisia IT Opportunity Finder

Plan de travail et architecture

Plateforme de recherche de stages PFE et d'emplois dans le domaine informatique en Tunisie.

Tous les objectifs du cahier des charges sont conservés : les 8 domaines, les 24 gouvernorats, les 4 types de filtres, le NLP, le clustering, l'analyse géographique, le dashboard complet et le matching. Seuls l'ordre de réalisation et le choix des outils ont été adaptés pour une débutante.

1. Architecture

L'architecture suit le flux de la section 12 du cahier des charges, avec deux couches en plus pour l'IA et l'interface.

```
Sources autorisées (sites tunisiens, pages carrières, CSV de secours)
        |
        v
1. DATA COLLECTOR    collectors/      un script par source
        |
        v
2. RAW DATA          data/raw/        fichiers bruts (CSV/JSON), jamais modifiés
        |
        v
3. CLEANING          pipeline/clean.py       HTML, doublons, dates, espaces
        |
        v
4. NORMALIZATION     pipeline/normalize.py   gouvernorat, domaine, type, technologies
        |            (utilise data/reference/*.json)
        v
5. DATABASE          SQLite (offers.db)
        |
   +----+-------------------+
   v                        v
6. NLP / ML             7. APPLICATION STREAMLIT
   ml/                      app/
   TF-IDF, K-Means          Accueil + recherche, Détail offre,
   clusters, matching       Dashboard, Clusters, Matching
   |                         ^
   +-- écrit les clusters --+
       dans la base
```

Pourquoi cette architecture convient à une débutante

Choix	Raison
Tout en **Python**	Un seul langage à maîtriser, et c'est celui de la Data Science
SQLite	Un simple fichier, rien à installer, et tu apprends le SQL
Streamlit	Une application web sans HTML, CSS ni JavaScript : filtres, listes et graphiques en quelques lignes
scikit-learn	TF-IDF, K-Means, PCA et similarité cosinus suffisent pour tout le cahier

Choix	Raison
Plotly	Graphiques interactifs, dont la répartition par région
Pas de Docker ni d'API séparée	Ce sont des bonus pour la fin, pas des prérequis

Le cahier parle d'une « plateforme web » : Streamlit en est une et reste déployable gratuitement plus tard.

Structure du dépôt

```
tunisia-it-opportunity-finder/
|-- data/
|   |-- raw/                    offres brutes
|   `-- reference/
|       |-- governorates.json   24 gouvernorats + villes/localités
|       |-- domains.json        métiers -> 8 domaines
|       `-- skills.json         technologies + synonymes
|-- collectors/                 un fichier par source
|-- pipeline/                   clean.py, normalize.py, load_db.py
|-- ml/                         text_cleaning.py, clustering.py, matching.py
|-- app/
|   |-- Home.py                 accueil + recherche + liste
|   `-- pages/
|       |-- 1_Dashboard.py
|       |-- 2_Clusters.py
|       `-- 3_Matching.py
|-- notebooks/                  exploration, choix de K, analyses
|-- docs/                       schémas, captures
|-- requirements.txt
`-- README.md
```

Base de données

Une table **offers** avec les champs de la section 13 du cahier : title, company, description, location, governorate, type, domain, skills, published_at, source, source_url, date_collected. En version 2, on ajoute une colonne **cluster_id**. Une seule table suffit ; elle pourra être découpée plus tard.

2. Le plan, étape par étape

Trois versions, chacune avec un résultat visible. Chaque étape commence par **ce qu'il faut comprendre avant de coder**. Le calendrier tient compte de la saison PFE (décembre à février).

Version 1 : centraliser et rechercher (sections 3 à 14, 18, 19) - cible : fin octobre / début novembre

Étape	À comprendre d'abord	À faire	Livrable
1. Cadrage 2-3 j	Le flux sources, base, application. Les règles de collecte (robots.txt, CGU, pause de 2-3 s)	Créer le dépôt GitHub, choisir 2 sources autorisées, dessiner le schéma	README initial et schéma
2. Référentiels 3 j	Ce qu'est un référentiel et pourquoi on normalise (section 14)	Remplir governorates.json (24 gouvernorats et leurs villes), domains.json (8 domaines), skills.json (technologies de la section 11 et synonymes)	3 fichiers JSON
3. Collecte 4-5 j	HTML de base, requests, BeautifulSoup. Un collecteur récupère, il ne nettoie pas	Un collecteur par source, sauvegarde avec source, source_url, date_collected. CSV manuel en secours	150 offres brutes ou plus
4. Exploration 2 j	Lire des données avant de les nettoyer	Notebook 01_exploration : valeurs manquantes, formats de lieux, titres variés	Liste des problèmes à corriger
5. Nettoyage et normalisation 5-6 j	Regex, dictionnaires, correspondance ville vers gouvernorat	« Sahloul - Sousse » et « Sousse, Tunisia » deviennent Sousse. Titres vers domaine. Détection Stage PFE / Emploi. Extraction des technologies. Suppression des doublons	Table offers propre dans SQLite
6. Application 5-6 j	Comment Streamlit relit le script à chaque action	Accueil avec les 4 filtres et le bouton Rechercher. Liste « 27 offres trouvées » (section 9), tri par date, page détail, bouton Postuler vers la source	Application de recherche en local
7. Dashboard v1 3-4 j	KPI et agrégations avec groupby	KPI (total, stages, emplois, entreprises, régions, domaines). Graphiques : par région, par domaine, stages vs emplois, technologies, évolution temporelle, croisement région x domaine	Page Dashboard

Test de fin de V1 : tu sais répondre dans l'application à « Combien d'offres AI & Data à Sousse ? » et « Quelle région propose le plus de stages PFE en développement ? ». Tu publies ensuite sur GitHub avec un README illustré.

Version 2 : NLP et clustering (sections 15, 16, 19) - cible : mi-novembre

Étape	À comprendre d'abord	À faire	Livrable
8. NLP 3-4 j	Un ordinateur lit des nombres, pas du texte. Mots vides et tokenisation en français et en anglais	Fusionner Titre + Description + Compétences, nettoyer (minuscules, ponctuation, stopwords FR/EN)	ml/text_cleaning.py

Étape	À comprendre d'abord	À faire	Livrable
9. TF-IDF 2-3 j	Un mot est important s'il est fréquent dans une offre mais rare dans les autres	Construire la matrice de caractéristiques	Notebook expliqué
10. K-Means 4-5 j	Les centroïdes, le choix de K (méthode du coude, coefficient de silhouette)	Tester K de 3 à 10, choisir K, lire les mots dominants et nommer les clusters (Python Backend, Frontend, AI/Data, Java Backend, Cybersecurity, DevOps...)	Clusters nommés
11. Visualisation 2-3 j	La réduction de dimension (PCA) pour afficher en 2D	Nuage de points des clusters, enregistrer cluster_id dans la base	Visualisation
12. Dashboard v2 3 j	Croiser une colonne ML avec les autres	Page Clusters : visualisation, mots clés par cluster, répartition des clusters par région	Dashboard complet (section 19)

Version 3 : matching profil et offres (section 3, point 10) - cible : fin novembre

Étape	À comprendre d'abord	À faire	Livrable
13. Matching 5-7 j	La similarité cosinus : deux textes sont proches si leurs vecteurs pointent dans la même direction	Saisir ses compétences ou coller son CV, vectoriser avec le même TF-IDF qu'en V2, classer les offres par score	Page Matching
14. Explication 2 j	Un score seul n'est pas parlant	Afficher les compétences que tu as et celles qui te manquent pour chaque offre	Matching expliqué
15. Finitions 3-4 j	Ce qui fait un bon dépôt	README avec démo (GIF), captures, résultats chiffrés, section « Limites et éthique de la collecte », déploiement sur Streamlit Community Cloud	Projet publié

Total : environ 8 semaines à un rythme raisonnable. Si tu es en retard, la V1 seule te donne déjà un projet présentable.

3. Correspondance avec le cahier des charges

Section du cahier	Où elle est traitée
3-4 Solution et objectifs candidat	Étapes 6 et 13
5 Périmètre (8 domaines)	Étape 2 (domains.json)
6 Types Stage PFE / Emploi	Étape 5
7 Zones géographiques (24 gouvernorats)	Étape 2 (governorates.json)
8-9 Accueil et liste des offres	Étape 6
11 Filtrage	Étape 6
12 Collecte	Étapes 3 et 5
13 Structure des données	Table offers

Section du cahier	Où elle est traitée
14 Nettoyage et normalisation	Étape 5
15 NLP	Étapes 8 et 9
16 Clustering	Étapes 10 et 11
18 Analyse géographique	Étapes 7 et 12
19 Dashboard	Étapes 7 et 12
Matching (ultérieur)	Étapes 13 et 14

Dans le cahier des charges, la numérotation saute les sections **10** et **17**. Suggestion : la 10 pour la page de détail d'une offre, la 17 pour la visualisation des clusters (PCA).

4. Règles pour bien apprendre

1. **Un seul script à la fois**, avec un test visible (un print ou un head()) avant de passer au suivant.
2. **Explore dans un notebook, puis déplace le code qui marche dans un fichier .py.**
3. **Commit à chaque étape terminée**, avec un message clair (feat: collecteur source 1).
4. **Écris dans le README ce que tu as appris à chaque étape.** C'est ce qui rend un dépôt convaincant pour un recruteur.
5. **Reste sur la V1 tant qu'elle n'est pas finie.** Les clusters ne valent rien sans données propres.

5. Par quoi commencer

Étapes 1 et 2, cette semaine :

- créer le dépôt avec la structure ci-dessus (fait) ;
- lister 2 ou 3 sources d'offres IT tunisiennes et vérifier leurs conditions d'utilisation (robots.txt et CGU) ;
- compléter et relire skills.json, governorates.json et domains.json.

Règles de collecte : ne pas collecter LinkedIn ni Indeed ; respecter robots.txt et les CGU ; attendre 2 à 3 secondes entre deux requêtes ; ne conserver que les informations publiques de l'offre et le lien vers l'annonce originale.